Ders 14 Genişletilmiş Notu: Dönem Projesi Sunumları (2. Gün)

Ders 14 Genişletilmiş Notu: Dönem Projesi Sunumları (2. Gün)

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Düzeni ve Sunumlardan Beklenenler
- 2. Birinci Sunum: Yazılım Evi İnsan Kaynakları Sistemi
- 3. İkinci Sunum: Medya Takip Şirketi Bilgi Sistemi (Katmanlı Mimari ve Mock'lu Testler)
- 4. Üçüncü Sunum: İnsan Kaynakları Sistemi (İki Muhasebe Programı)
- 5. Dördüncü Sunum: Seyahat Acentesi Biletleme Sistemi (C++ ve GoogleTest)
- 6. Beşinci Sunum: Seyahat Acentesi (Çevik Model Tartışması ve Tasarım Kalıpları)
- 7. Altıncı Sunum: İnsan Kaynakları Sistemi (include/extend ve Etkinlik Diyagramı Soruları)
- 8. Yedinci Sunum: Medya Takip Sistemi (Abonelik ve Onay Akışı)
- 9. Sekizinci Sunum: İnsan Kaynakları Sistemi (Başvuru Odaklı Tasarım)
- 10. Dokuzuncu Sunum: İnsan Kaynakları Sistemi (Milestone ve Projeye Özgü Risk)
- 11. Onuncu Sunum: Seyahat Acentesi (PostgreSQL ve PyTest)
- 12. Sunumlardan Çıkan Ortak Dersler

1. Dersin Düzeni ve Sunumlardan Beklenenler

Bu ders, dönem projesi sunumlarının ikinci ve son günüdür. Bir önceki hafta birkaç grup sunum yapmıştı; bu derste geri kalan bütün gruplar sundu. Hoca derse sunum sırasını belirleyerek başladı. Hazır olan gruplar söz aldı ve sıra bunlara göre yazıldı. İlk dört sunumdan sonra kısa bir ara verildi, sonra kalan gruplar devam etti. Toplam on grup sunum yaptı. Bu derste yeni bir konu anlatılmadı. Akademik içerik, grupların sunduğu analiz ve tasarım belgeleri ile hocanın bunlara yönelttiği sorulardan oluşur.

Projeler üç vaka üzerine kuruluydu:

- **Yazılım evinin insan kaynakları sistemi:** Çalışanların bilgilerini tutmak, onları uygun projelere atamak, proje bittiğinde başka projeye aktarmak, aktarılamayanların tazminatını hesaplayıp işten çıkarmak ve maaşları muhasebe programlarıyla hesaplamak.
- **Medya takip şirketinin bilgi sistemi:** Müşterilerin istediği konularda medyada çıkan haberleri izleyip rapor ve anket hizmeti sunmak.
- **Seyahat acentesinin bilet ve tatil sistemi:** Otobüs, uçak ve tren biletleri ile otel rezervasyonlarını satmak, müşteriye harcamasına göre puan kazandırmak.

Her grubun sunumu benzer bir sırayı izledi: alan tanımı, kabuller ve kısıtlar, iş zaman çizelgesi (Gantt diyagramı), ekip organizasyonu, risk tablosu, kullanım senaryoları (use case) ve senaryo belgesi, izlenebilirlik tablosu, kavramsal ve tasarım sınıf diyagramları, sıralama (sequence), etkinlik (activity) ve durum (state) diyagramları, tasarım kalıpları (design patterns) ve birim testleri (unit tests). Grupların programlarını ayrıca dersin asistanına gösterdiği anlaşıldı. Bu yüzden hoca çoğu grupta arayüz gösterimini atladı ve sorularını belgeler ile kavramlar üzerinde topladı.

Hoca sunumların başından itibaren bazı tekrarları kısalttı. Alan tanımı her grupta aynı vakadan geldiği için onu dinlemek istemedi:

> **Hocanın vurgusu:** Alan tanımını ve vakada zaten verilen kabulleri biliyoruz. Sadece sizin eklediğiniz kabuller ve kısıtlar, yani size özgü şeyler varsa onları söyleyin.

Hocanın sunumlar boyunca tekrar tekrar sorduğu sorular şunlardı: hangi süreç modelini seçtiniz, Gantt diyagramında milestone var mı, risklerin önlemi nedir, use case belgeniz nerede, iki sınıf diyagramından hangisi hangi aşamada kullanılır, etkinlik diyagramı ne için çizilir, include ve extend oklarının yönü nedir, tasarımda kalıp kullandınız mı. Bu sorular ve cevapları aşağıda her sunumun içinde, sonunda da "Örnek Sorular" bölümünde toplanmıştır.

2. Birinci Sunum: Yazılım Evi İnsan Kaynakları Sistemi

2.1 Amaç, Kabuller ve Kısıtlar

İlk grup, yazılım projeleri yürüten bir şirketin insan kaynakları departmanı için bir sistem tasarladı. Amaç, şirket içindeki çalışanların en uygun projede görevlendirilmesini sağlamaktı. Grup üyeleri yönetici, analist, yazılımcı ve tasarımcı rollerine dağılmıştı, ama bütün aşamalarda birlikte çalıştıklarını söylediler.

Grubun kabulleri şunlardı:

- Her projede en az 3, en çok 5 çalışan bulunur.
- Çalışan pozisyonları yönetici, analist, tasarımcı ve programcıdır. İleride yeni pozisyon eklenebilir.
- Sisteme yeni bir çalışan kaydedildiğinde proje ataması pozisyonuna göre yapılır.
- Her projede bazı pozisyonların bulunması zorunludur, diğerleri isteğe bağlıdır.
- Bir proje bittiğinde çalışan için uygun pozisyon varsa başka projeye atanır, yoksa işten çıkarılır.
- Projeler sistemde önceden tanımlıdır, yeni proje eklenmez. Proje bitene kadar hiçbir çalışan projeden çıkmaz.
- Tazminat hesabı için başka bir şirketin web servisi kullanılır. Maaş hesabı için iki farklı program vardır ve çalışana uygun program iş başlangıcında seçilir.

Grubun iki kısıtı vardı. Birincisi, bütün modülleri yapmaya yetecek zaman yoktu; açık kalan noktalar proje tesliminden sonraya bırakıldı. İkincisi, test için ayrı kişi yoktu; testleri ekibin kendisi yaptı.

2.2 Proje Takvimi

Takvim şöyleydi: proje planlamaya iki gün (takvim, görev dağılımı ve hedef belirlendi), gereksinim analizine iki hafta (kabul ve kısıtlar, yazılım dili seçimi, use case diyagramı), tasarıma bir hafta, kodlama ve kalan diyagramlara on gün ayrıldı, en sonda test yapıldı. Grup bu aşamaları şelale (waterfall) süreç modeline göre sıraladığını söyledi.

2.3 Use Case Diyagramı ve Senaryolar

Use case diyagramında iki aktör vardı: insan kaynakları (İK) personeli ve çalışan (personel). Sistemin bütün özelliklerine İK erişiyordu; personel hiçbir senaryoda birinci aktör değildi. Beş temel kullanım senaryosu vardı: personel bilgilerini görüntüleme, işe alma, proje bitirme, yeni pozisyon (çalışan grubu) ekleme ve proje bilgilerini görüntüleme.

Grup bunlardan birkaçını senaryo belgesi olarak anlattı:

- **Proje bitirme:** Birinci aktör İK personelidir, etkilenen aktör çalışandır. Ön koşul, proje için ayrılan sürenin bitmiş olmasıdır. Ana akışta İK personeli ana ekranda "proje işlemleri" butonuna tıklar, projeleri listeler, kapatılacak projeyi seçip "projeyi kapat" der. Projedeki çalışanlar uygun başka bir projeye atanır. Alternatif akışta uygun proje yoksa çalışanlar işten çıkarılır.
- **Proje bilgilerini görüntüleme:** İK personeli "proje işlemleri" ve ardından "projeleri listele" butonuyla bütün projeleri görür.

- **İşe alma:** İK personeli ana ekrandaki iki butondan (personel işlemleri ve proje işlemleri) "personel işlemleri"ni seçer, çalışan bilgilerini doldurur, seçilen maaş programıyla maaşı hesaplar ve kaydeder. Çıkan ekranda çalışanın atandığı proje ve maaş bilgisi görünür. Alternatif akışta eksik veya hatalı bilgi varsa düzeltme uyarısı çıkar.

2.4 İzlenebilirlik ve Risk Tablosu

İzlenebilirlik tablosunda gereksinimler (personel alımı, proje bitirme, yeni çalışan grubu gibi İK'nın yapması gereken işler) ile bu gereksinimi karşılamak için birlikte çalışması gereken sınıflar eşleştirildi. Örneğin personel alımı personel, proje ve muhasebe sınıflarıyla etkileşir.

Risk tablosunda üç örnek verildi:

Risk	Önem derecesi	Olasılık
Personel kayıtlarının silinmesi	Yüksek	Düşük
Personelin yanlış projeye atanması	Orta	Düşük
Personel yetersizliği (projede 3'ten az çalışan kalması)	Orta	Orta

Hoca burada iki şey hatırlattı. Birincisi, tablolarda riskin önlemi de verilmelidir. İkincisi, önem derecesi risk olasılığı ile etkisinin çarpımından ortaya çıkar. Grup, önlemleri uygulama içinde gösterdikleri için tabloya eklemediklerini söyledi; hoca tabloya da eklenebileceğini belirtti.

2.5 Sınıf Diyagramları

Grup ilk sınıf diyagramını "tasarım sınıf diyagramı" başlığıyla gösterdi. Hoca bunun gerçekten tasarım diyagramı mı yoksa kavramsal sınıf diyagramı mı olduğunu sordu. Grup diyagramın kavramsal olduğunu, başlığı yanlış yazdıklarını kabul etti.

Kavramsal diyagramda bir ana **Personel** sınıfı ve onun alt sınıfları olarak programcı, tasarımcı, analist ve yönetici vardı. Bir **Muhasebe** sınıfı tazminat ve maaş hesaplarını içeriyordu. Bir **Proje** sınıfı da projenin en az çalışan sayısı gibi bilgileri tutuyordu.

Tasarım sınıf diyagramında aynı sınıflara metotlar eklenmişti. Personel sınıfında çalışan ekleme, çalışan kimliği (ID) üretme ve otomatik proje atama; Proje sınıfında bütün projeleri gösterme ve projeyi bitirme; Muhasebe sınıfında maaş hesaplama ve tazminat hesaplama metotları vardı.

2.6 Sıralama, Etkinlik ve Durum Diyagramları

İki sıralama diyagramı çizilmişti: personel ekleme ve proje bitirme. Personel eklemede Personel sınıfındaki bir ekleme metodu çalışıyor, diğer bütün metotlar bunun içinden çağrılıyordu.

Etkinlik diyagramları personel ve proje bilgilerini görüntüleme, pozisyon ekleme, personel ekleme ve maaş/tazminat hesabı için çizilmişti. Öğrenci diyagramları tek tek anlatmak isteyince hoca durdurdu ve şu soruyu sordu:

Soru: Etkinlik diyagramı özellikle neyi ortaya koymak için çizilir? **Cevap (öğrenci):** Etkinliğin akışının nasıl gerçekleştiğini, sırasını ve koşullarını anlatmak için.

Hoca cevabı tamamladı: Etkinlik diyagramı ayrıca etkinliğin farklı ortamlarda nasıl yürüdüğünü gösterir. Paralel yürüyen farklı aktivitelerin farklı kişileri ve ortamları nasıl etkilediğini, paralel akışların nerede ayrılıp nerede birleştiğini göstermek istediğimizde etkinlik diyagramı en uygun araçtır.

> **Hocanın vurgusu:** Etkinlik diyagramı yalnızca adım sırası değildir. Farklı ortamlarda (kişi, birim) yürüyen paralel akışları ve bunların birleştiği yerleri göstermek için kullanılır.

Durum diyagramında personel ve proje için ayrı ayrı iki diyagram vardı. Her ikisinde de nesnenin aktif ve pasif durumları ve aralarındaki geçişler gösterilmişti.

2.7 Birim Testleri

Testler kişi başına iki test olacak şekilde paylaşılmıştı. Örnekler:

- **Giriş ekranı testi:** Ana ekrandaki "personel işlemleri" butonuna basıldığında ilgili ekranın açılıp açılmadığını kontrol eder.
- **Veritabanı bağlantı testleri:** Biri personel işlemleri, biri proje işlemleri için olan iki SQL tabanlı veritabanına bağlantının doğru kurulup kurulmadığını kontrol eder.
- **Boş alan testi:** Personel eklerken bir metin kutusu boş bırakılırsa uyarı dönüp dönmediğini kontrol eder.
- **Kimlik ve çalışan oluşturma testleri:** Kimliğin doğru aralıkta üretilip üretilmediğini kontrol eder.

Hoca testlerin kullanılan araçla çalıştırıldığını görmek istedi. Grup on test dosyasının hepsini tek seferde çalıştırdı ve hepsinin başarılı geçtiğini gösterdi.

3. İkinci Sunum: Medya Takip Şirketi Bilgi Sistemi (Katmanlı Mimari ve Mock'lu Testler)

3.1 Alan, Kabuller ve Fizibilite

İkinci grup, medya takip şirketi vakasını bir demo uygulama olarak geliştirmişti; raporları bu demonun raporu gibi hazırlanmıştı. Amaç, şahıs şirketlerine ve daha büyük şirketlere istedikleri konularda medyada neler olduğunu göstermekti. Kabul kriterlerinde müşterinin iki temel işi vardı: **rapor talebi** oluşturmak ve **anket talebi** oluşturmak. Diğer modüller bu iki modülün çalışması için gerekliydi; ödeme modülü de bunları destekliyordu.

Kayıt akışında müşteri ya şahıs ya da şirket olarak kayıt oluyordu. İki kayıt tipinde farklı alanlar dolduruluyordu, ama ortak ve asıl işlevi belirleyen **rapor tipi** kayıt sırasında alınıyordu. Hoca bu noktada gereksinimleri use case diyagramı üzerinden anlatmalarını istedi.

Teknik fizibilitede araçlar şöyle seçilmişti: arayüz için Angular, veritabanı için PostgreSQL, uygulama tarafında Java ve Spring. Seçim gerekçeleri araçların açık kaynaklı olması, ekibin bunlara aşina olması ve popüler oldukları için internette kaynak bulmanın kolay olmasıydı. Donanım fizibilitesi için işlemci, grafik, RAM, depolama ve işletim sistemi tablosu hazırlanmıştı; hoca bunu geçmelerini istedi.

3.2 Proje Planı ve Risk

Proje planında planlama ve ön analize yaklaşık 11 gün, tasarıma yaklaşık 6 gün, geliştirmeye 1 hafta, teste 1 hafta ayrılmıştı. Ekip ikiye bölünmüştü: bir kısım analiz ve tasarımı, diğer kısım gerçekleme, sınama ve bakımı üstlendi. Veri kaynakları anketler, veritabanları, medya organları ve iç verilerdi.

Risk tablosundan örnek olarak **veri kaybı** verildi: risk önceliği orta, etkisi yüksek, gerçekleşme olasılığı orta, yönetim stratejisi "azaltıcı" (risk azaltma).

3.3 Use Case Diyagramı ve Senaryo Belgesi

Üç aktör vardı: müşteri, şirket yöneticisi ve çalışan.

- **Şirket yöneticisi:** kaynak ekler, siler, günceller. Bunun için kaynaklara erişmesi gerekir.
- **Müşteri:** sisteme giriş yapar (kayıtlı değilse önce kayıt olur), rapor görüntüler, rapor görüntülerken sorun yaşarsa çalışanla iletişime geçer, yeni konu talebinde bulunur (bu talebin içinde anket talebi seçeneği de vardır), ödeme yapar ve eski ödemelerini görüntüler.
- **Çalışan:** sisteme giriş yapar, kaynaklara erişir, müşterilerle iletişime geçer, anketörlere erişir ve onlara iş verir.

Hoca senaryo belgesinden birini görmek istedi. Grup **müşterinin sisteme girişi** senaryosunu gösterdi. Ön koşul sistemin arayüzüne ulaşılmış olması, son koşul sisteme sorunsuz giriş yapılmış olmasıdır. Ana senaryoda müşteri e-posta ve parolasını girip giriş butonuna basar. Alternatif senaryoda girilen e-posta adresi bulunamazsa sistem kayıt oluşturmayı önerir. Öneri

kabul edilirse müşteri şahıssa ad, soyad, e-posta, telefon, parola, doğum yeri, doğum tarihi, adres, meslek ve rapor tipi; şirketse benzer ama birkaç alanı farklı bir bilgi seti alınır. Müşteri "kaydet" butonuna basınca kayıt tamamlanır.

3.4 Alan Modeli ve Backend Katmanları

Alan modelinde (kavramsal sınıf diyagramı) şu ilişkiler anlatıldı:

- Bir temel sınıfa Authentication, Response, RegisterRequest ve User sınıfları bileşim (composition) ile bağlıdır.
- User sınıfına BaseCustomer ve Demand sınıfları bileşim ile bağlıdır. (Öğrenci önce "aggregation" deyip "composition" diye düzeltti.)
- BaseCustomer'dan kalıtımla (inheritance) PersonalCustomer ve CompanyCustomer türer.
- Source ve CreateSourceRequestInfo sınıfları SourceType'a bileşimle bağlıdır.
- PersonalCustomer, CompanyCustomer, PersonalRegisterRequest ve CreateReportDemandRequest sınıfları ReportPeriodType'a bileşimle bağlıdır.

Backend üç katmandan oluşuyordu:

- **Service sınıfları:** iş mantığı (business logic) işlemleri.
- **Controller sınıfları:** kullanıcının istekleri ve bunlara dönülen cevaplar.
- **Repository sınıfları:** saklama ve depolama işlemleri.

Örneğin rapor için ReportService, ReportController ve ReportRepository vardı; diğer alanlarda da aynı yapı tekrar ediyordu.

İzlenebilirlik tablosunda sütunlar senaryo, amaç, öncelik ve sınıftı. Örnek: "sisteme giriş" senaryosunun amacı müşterinin sisteme kaydı, önceliği 1, ilgili sınıfı Müşteri. Hoca tabloya bir kez daha baktı.

3.5 Etkinlik, Sıralama ve Durum Diyagramları

Etkinlik diyagramları müşteri, çalışan ve yönetici için ayrı çizilmişti. Müşteri diyagramının akışı şöyleydi:

```
[Başla] -> Arayüze giriş isteği
  karar: Kaydı var mı?
    hayır -> Bilgileri gir, kayıt ol
    evet  -> Sisteme giriş
  karar: Yeni talep oluşturmak istiyor mu?
    hayır -> karar: Önceki raporları görmek istiyor mu?
               evet  -> Önceki raporları görüntüle
               hayır -> Ana menü
    evet  -> Konu talebinde bulun
             karar: Anket istiyor mu?
               evet -> Anket içeriğini doldur
             -> Ödeme yap
  karar: Ek işlem yapacak mı?
    evet  -> istenen işleme dön
    hayır -> [Bitiş]
```

Sıralama diyagramı örneği **rapor görüntüleme** idi. Müşteri ana sayfadan rapor görüntüleme isteğini sisteme iletir, raporlar listelenir. Alternatif olarak müşteri belirli bir raporu seçer, sistem raporu bulup iletir. Rapor görüntülenemezse müşteri iletişim sayfasını açar, şikâyet bilgilerini girer, sistem bir iletişim kaydı oluşturur ve müşteriye onay mesajı gösterilir.

Durum diyagramı örneği şirket yöneticisi üzerindendi: yönetici sistemdeyken güncelleme, silme veya ekleme için bir kaynak seçer; sistem "kaynak erişimde ve değişiklik aşamasında" durumuna geçer. Kayıt başarılıysa yönetici tekrar "sistemde" durumuna döner, hata olursa sistem "hata sayfası" durumuna geçer. Yönetici çıkış yaparsa son duruma gidilir.

3.6 Tasarım Sınıf Diyagramı ve Builder Kalıbı

Tasarım sınıf diyagramında kavramsal modele metotlar eklenmişti. Model (domain) sınıflarında yalnızca getter, setter ve builder metotları vardı. Grup **Builder** tasarım kalıbını, modeli üretirken daha "nokta atışı" ve kontrollü bir üretim sağladığı için seçtiğini söyledi.

Asıl mantık repository, service ve controller katmanlarındaydı. Örneğin Source için: SourceRepository veritabanı üzerinde işlem yapar, SourceService veritabanından çekilen veriyi işler ve günceller, SourceController bu kaynak üzerindeki isteklerin kabul edilip edilmeyeceğini yönetir. Grup bu katmanları birbirine bileşimle bağladığını söyledi. Gerekçesi şuydu: DemandService içinde DemandRepository bulunmak zorundadır, çünkü kaynaklar repository'dedir; controller'ın da kendi işlem birimi yoktur, servisten hizmet almak zorundadır.

> **Ders sonrası not:** Grup controller, service ve repository arasındaki ilişkiyi bileşim olarak çizdi. Spring gibi çatılarda bu nesneler büyük olasılıkla bağımlılık enjeksiyonuyla (dependency injection) dışarıdan verilir; servis, repository nesnesinin yaşam döngüsünün sahibi değildir. Bu yüzden ilişkinin ilişki (association) ya da bağımlılık (dependency) olarak gösterilmesi daha doğrudur.

3.7 Mock Kullanan Birim Testi

Grup birim testinin amacını şöyle açıkladı: Yazdığımız metodu test etmek istiyoruz. Bunun için metodun dış bağımlılıklarını "mış gibi yapıyoruz", yani mock'luyoruz ve yalnızca ilgili metodun çalışmasını test ediyoruz.

Java ile yazılmış örnekte bir rapor talebi isteği (request) ve bir kullanıcı oluşturuluyordu. Test edilen metot `createReportDemand` idi. Bu metoda istek ve kullanıcı verildiğinde, repository katmanının `save` metodunun tam bir kez çağrılıp çağrılmadığı `verify(..., times(1))` ile doğrulanıyordu. Grup testin çalıştırılmış hâlini de gösterdi.

Sunumun sonunda grup, üyelerinin bir kısmının dersin diğer şubesinden olduğunu belirtti. Hoca bunu not etti ve diğer şubeden olan herkesin bunu belirtmesini istedi. Ardından kısa bir ara verildi.

4. Üçüncü Sunum: İnsan Kaynakları Sistemi (İki Muhasebe Programı)

4.1 Kabuller ve Gruba Özgü Ekleme

Üçüncü grup yine yazılım evinin insan kaynakları sistemini sundu. Sistem çalışan bilgilerini tutuyor, çalışanları uygun projelere yerleştiriyor, işten çıkarılanların tazminatını hesaplıyor ve maaşlar için farklı muhasebe programları kullanıyordu. Grup vakadaki kabulleri okumaya başlayınca hoca yalnız gruba özgü olanları söylemelerini istedi. Grubun eklediği kabul şuydu:

- Çalışan 2010'dan önce işe başladıysa maaşı **A muhasebe programıyla**, 2010'dan sonra başladıysa **B muhasebe programıyla** hesaplanır.

4.2 Süreç Modeli Sorusu ve Ekip

İş planında fizibilite analizi, proje planı, kullanım senaryolarının belirlenmesi, nesneye dayalı modelleme ve diyagramlar, projenin Python ile gerçeklenmesi, yeniden düzenleme (refactoring), birim testi ve raporlama vardı. Hoca Gantt diyagramını nerede çizdiklerini sordu (Google Sheets). Sonra şu soruyu sordu:

Soru: Hangi süreç modelini kullandınız? **Cevap:** Öğrenci önce bir şablon bulup kendi planlarını Gantt diyagramında gösterdiklerini söyledi. Hoca soruyu netleştirdi: "Bir sürü süreç modeli gördük; model olarak hangisini seçtiniz?" Grup şelale (waterfall) modelini kullandıklarını söyledi.

Ekip organizasyonunda proje yöneticisi, kalite analisti, iş analisti, sistem mimarı ve tasarımcısı ile yazılım geliştirici rolleri vardı.

4.3 Risk Tablosu

Grup yedi risk belirlemiş, her biri için açıklama, etki, olasılık ve önlem yazmıştı:

Risk	Etki	Olasılık	Önlem
Yeni çalışanın bilgilerinin sisteme yanlış girilmesi (yanlış projeye atanma)	Büyük	Düşük	Bilgilerin doğru girildiğinin iki kez kontrol edilmesi
Çalışanların projeler arasında yanlış aktarılması (performans düşüşü)	Büyük	Düşük	Yöneticilerin çalışan performansını izleyip zamanında müdahale etmesi
Tazminat hesaplamalarında hata	BELİRSİZ	Orta	Hesapların iki taraflı kontrolü
Maaş hesaplamalarında hata (çalışanlar arasında memnuniyetsizlik)	Çok büyük	Düşük	İki taraflı doğruluk kontrolü
Maaş için yanlış programın kullanılması (A yerine B)	BELİRSİZ	Orta	İK personelinin doğru programın kullanıldığını kontrol etmesi

4.4 Kullanım Senaryosu ve Use Case Belgesi

Grup **çalışan bilgilerinin sisteme girilmesi** senaryosunu anlattı:

- **Birinci aktör:** İK personeli. **Etkilenen aktör:** çalışan.
- **İlgililerin beklentileri:** İK personeli çalışanın özlük bilgilerini alıp sisteme işlemek ister; çalışan özlük bilgilerini ve atandığı projeyi sistemde görmek ister.
- **Ön koşul:** İK personeli kendi kullanıcı bilgileriyle sisteme girmiştir ve projede en çok çalışan sayısı aşılmamıştır.
- **Son koşul:** Çalışanın özlük ve proje bilgileri sistemde görüntülenebilir.
- **Ana akış:** (1) İK personeli yeni çalışandan özlük bilgilerini ister. (2) Çalışan bilgilerini eksiksiz paylaşır. (3) İK personeli bilgileri sisteme kaydeder. (4) Sistem çalışanı uygun projeye atar.
- **Alternatif akış:** İkinci adımda çalışan yurt dışındaki bir üniversiteden mezunsa ilgili üniversiteden denklik belgesinin kontrolü gerekir.

Use case diyagramında üç aktör vardı. İnsan kaynakları maaş hesaplama, çalışan alma ve çalışanı işten çıkarma işlerini; yönetici planlama, çalışan bilgisi görüntüleme, projeye çalışan atama, proje oluşturma ve proje bitirme işlerini yapıyordu; çalışan ise proje geliştirme sürecine katkı veriyordu. Use case'ler arasında include ilişkileri vardı: maaş hesaplama, işten çıkarma ve çalışan bilgisi görüntüleme için önce çalışanın sisteme eklenmiş olması gerekiyordu.

Hoca başka alt senaryoları ve **use case belgesini** sordu. Öğrenci soruyu ilk anda anlamadı. Hoca açıkladı:

> **Hocanın vurgusu:** Kullanım senaryosunu diyagram olarak gösteriyorsunuz; bir de onun açıklamalı tanımı, yani use case belgesi olmalı.

Grup bir önceki slayttaki adım adım senaryonun bu belge olduğunu gösterdi; diğer senaryoları rapora koyduklarını söylediler.

4.5 İzlenebilirlik ve Sınıf Diyagramı

İzlenebilirlik tablosunun amacı, yazılım geliştirme sürecinde gereksinimlerin gerçeklenmesini izlemekti. Tabloda yönetim için proje oluşturma, bitirme, personel atama ve ekleme; İK çalışanı

için çalışan bilgilerinin girilmesi, maaş ve tazminat hesabı gibi gereksinimler vardı.

Sınıf diyagramı sistemin statik yapısını UML notasyonuyla gösteriyordu: sınıflar, aralarındaki ilişkiler, öznitelikleri (attribute) ve davranışları (metot). Örnekler:

- Programcı, analist, tasarımcı ve test uzmanı (tester) sınıfları Çalışan sınıfından kalıtılır.
- Maaş ile muhasebe programı arasında sıkı bir ilişki vardır: maaşı muhasebe programı hesaplar, bir program birden fazla maaş hesaplayabilir. Tazminat uygulaması ile tazminat arasında da benzer bir ilişki vardır.
- Yönetici çalışanı terfi ettirebilir (promote), atayabilir ve rapor alabilir.
- Çalışanın öznitelikleri kimlik, ad, soyad, telefon, unvan ve göreve başlama tarihidir; personel ekleme, silme ve bilgi getirme gibi metotları vardır.

4.6 Durum, Etkinlik ve Sıralama Diyagramları

Grup "maaş hesaplama durum diyagramı" başlığıyla şu akışı anlattı: İK personeli sisteme girer, çalışan bilgilerine erişir, işe başlama tarihini ve çalışılan gün sayısını hesaplar. Çalışan 2010'dan önce başladıysa A programı, sonra başladıysa B programı açılır. Maaş hesaplanır, ödeme için muhasebeye bildirilir ve İK personeli çıkış yapar.

> **Ders sonrası not:** Bu diyagram "durum diyagramı" diye sunuldu, ama anlatılan şey bir aktörün yaptığı işlerin sırasıdır. Bu büyük olasılıkla bir etkinlik diyagramıdır. Durum diyagramı tek bir nesnenin (ör. maaş kaydı: hesaplanmadı, hesaplandı, muhasebeye bildirildi) durumlarını ve geçişlerini gösterir.

Çalışan ekleme etkinlik diyagramında İK personeli sisteme girer, çalışan ekleme sayfasını açar, bilgileri girip "personel ekle" butonuna tıklar. Bir karar düğümünde yanlış veya eksik bilgi varsa geri dönülür; yoksa çalışan eklenir ve uygun projeye atanır.

Tazminat hesabının sıralama diyagramında sistem ile tazminatı hesaplayan dış web servisi arasındaki ilişki gösteriliyordu. Her personel ayrıldığında bir döngü (loop) çalışıyordu: İK sisteme girer, sistem web servisine login olur, servis doğrulamayı döndürür ve hesap yapılır. Hoca burada tasarım kalıbı kullanıp kullanmadıklarını sordu; grup kullanmadığını söyledi.

4.7 Birim Testleri

Testler kişi başına iki tane olacak şekilde dağıtılmıştı. Gösterilen örnekte yazılıma girişte kullanılan kullanıcı adı ve parolanın doğrulanması test ediliyordu: doğru bilgilerle `assertTrue`, yanlış bilgilerle `assertFalse` beklentisi kontrol ediliyordu.

5. Dördüncü Sunum: Seyahat Acentesi Biletleme Sistemi (C++ ve GoogleTest)

5.1 Proje Bilgi Formu ve İhtiyaçlar

Dördüncü grup seyahat ve tatil acentesinin biletleme sistemini sundu. Raporlarını, yazılım projelerinin nasıl raporlandığına dair internette buldukları başlık yapısına göre düzenlemiş ve her projede bulunan bir **proje bilgi formunu** kendi projelerine uyarlamışlardı.

Projenin amacı iş yönetimini kolaylaştırmak, verimi ve müşteri memnuniyetini artırmaktı. Grup ihtiyaçları iki taraftan ele aldı. Müşteri tarafında elle yürüyen bir sistemde puan biriktirme imkânının olmaması bir dezavantajdı. Şirket tarafında müşteriye yönelik ilgi çekici kampanyalar düzenlenememesi bir dezavantajdı. Bu yüzden dijital bir sistem gerekiyordu. Hoca burada da yalnız gruba özgü kısıt ve kabulleri anlatmalarını istedi.

5.2 Gantt Diyagramı ve Süreç Modeli

İş paketleri dört ana başlık altında toplanmıştı: analiz, tasarım, kodlama ve sınama. Hoca süreç modelini sordu; grup şelale (waterfall) modelini, yani birbirini izleyen iş paketlerini kullandığını söyledi. Ekip organizasyonunda herkesin eşit görev aldığı bir dağılım vardı.

5.3 Use Case Diyagramı

Üç aktör vardı: müşteri, acente yetkilisi ve yönetim. Akış şöyleydi:

- **Bilgi girişi:** Müşteri tatil bilgisi ya da bilet bilgisi girer.
- **Sisteme işleme:** Girilen bilgi, duruma göre bilet satış sistemine ya da tatil sistemine işlenir. Bu adımı acente yetkilisi de görür.
- **Sistem kontrolü:** Bilet fiyatı, stok ve uygun otel kontrolü yapılır; seçenekler belirlenir.
- **Karar verme ve ödeme:** Müşteri karar verir ve öder. Ödeme kredi kartıyla ya da daha önceki alışverişlerden kazanılan puanlarla yapılabilir.
- **Sistem bilgilendirmesi:** Ödemeden sonra acente sistemi, ardından otel ya da ulaşım firmasının sistemi bilgilendirilir.
- **Yönetim:** Puan oranlarında değişiklik yapabilen aktördür; sisteme üye ulaşım firmaları ve oteller hakkında değişiklikleri de yönetir.

Hoca use case belgesini görmek istedi. Grup bilgi girişi, sisteme işleme, sistem kontrolü, karar verme ve ödeme, sistem bilgilendirmesi senaryolarını tek tek yazdıklarını gösterdi.

5.4 Sınıf Diyagramı

Sınıf diyagramında yaklaşık yedi sınıf vardı. Grup sınıflar arasındaki ilişkileri diyagrama eklemeyi unuttuklarını kabul etti. Aralarında kalıtım yoktu; en büyük sınıf olan **TravelAgency** ile diğer sınıflar arasında bileşim ilişkisi vardı. Sağdaki **TicketInfo** sınıfı metot içermeyen, yalnızca bilgi tutan bir yapıydı (structure).

5.5 Sıralama Diyagramı

Hoca “üç tane sınıf mı var, neyin sıralama diyagramı bu?” diye sordu. Grup diyagramın müşterinin bilet alma akışını gösterdiğini söyledi. Akış şöyleydi:

```
Müşteri -> TravelAgency : bilet iste (tarih)
TravelAgency -> Şirket  : bu tarihe uygun bilet var mı?
Şirket -> TravelAgency  : bilet bilgileri
TravelAgency -> Müşteri : bilet bilgilerini göster
Müşteri -> TravelAgency : rezervasyon iste
TravelAgency -> Şirket  : rezervasyon talebi
Şirket -> TravelAgency  : onay
TravelAgency            : alışverişten gelen puanları hesapla
TravelAgency -> Müşteri : puanlar hesaba eklenir
Müşteri -> TravelAgency : önceki puanları kullan / kullanma
TravelAgency -> Müşteri : fiyat bilgisi
Müşteri -> TravelAgency : onay
```

5.6 Durum ve Etkinlik Diyagramları

Durum diyagramı biletin durumlarını gösteriyordu:

```
[Requested] --> [Available]    (bu tarih aralığında bilet var)
[Requested] --> [Unavailable]  (bilet yok)
[Available] --> [Reserved]     (müşteri rezerve etti)
[Reserved]  --> [Paid]         (ödeme alındı)
```

Etkinlik diyagramında müşteri ya biletlere bakabiliyor ya da satın alma yapabiliyordu. Bilet alınacaksa önce uygun olup olmadığına bakılıyor, uygunsa müşteri puanlarını kullanmayı ya da tam fiyat ödemeyi seçiyordu. Müşterinin puanı sıfırsa hata veriliyor, değilse yeni fiyat belirlenip ödeme alınıyordu. Ödemeden sonra puanlar müşterinin hesabına ekleniyor ve bilet veriliyordu.

Hoca burada bir eksik gördü:

Soru: Bu işlerin ortamlarını belirtmemişsiniz; hangisi hangi ortamda oluyor, öyle bir şeyiniz var mı? **Cevap:** Grup ortam ayrımı yapmadıklarını söyledi.

Bu soru, hocanın birinci sunumda söylediği “etkinlik diyagramı farklı ortamlardaki akışları gösterir” vurgusunun uygulamasıdır.

5.7 Birim Testleri (GoogleTest)

Hoca arayüzü görmek istemedi, testleri sordu. Grup C++ için Google’ın sunduğu **GoogleTest** çatısını kullanmıştı.

- **Birinci test:** Var olan bir biletin bilgilerinin müşteriye doğru verilip verilmediğini kontrol eder. Bilete rastgele değerler verilir, bilet bilgisini getiren fonksiyon çağrılır, dönen bilgiler saklanır ve beklenen değerlerle eşit olup olmadığına bakılır.
- **İkinci test:** Müşteri sistemde bulunmayan bir şirketten bilet isterse bir istisna (exception) fırlatılması gerektiğini kontrol eder.

6. Beşinci Sunum: Seyahat Acentesi (Çevik Model Tartışması ve Tasarım Kalıpları)

6.1 Proje Tanımı ve Kabuller

Beşinci grup da seyahat acentesi vakasını sundu. Acente tatil ve ulaşım hizmeti veriyordu. Ulaşım için bir dizi otobüs, havayolu ve demiryolu firmasının biletleri satılıyor, ileride başka firmaların eklenmesi hedefleniyordu. Tatil için çeşitli ülke ve bölgelerdeki otellerle anlaşma yapılmıştı. Müşteriler hizmetleri kullandıkça puan kazanıyor ve bu puanları sonraki tatil ve ulaşım alımlarında kullanabiliyordu. Puan kuralı şöyleydi: havayolu biletlerinde ücretin %1’i, tatillerde ücretin %2’si puan olarak geri veriliyordu.

Grup kabulleri ve kısıtları aynı başlık altında toplamıştı; örneğin müşteriler sistemi doğrudan kullanamıyordu (sistemi acente görevlisi kullanıyordu). Hoca bunu düzeltti:

> **Hocanın vurgusu:** Kabuller ve kısıtlar ayrı yazılır. Çünkü kabul ayrı, kısıt ayrı.

6.2 Çevik Model Seçildiyse Gantt Diyagramı Nasıl Olmalı?

Gantt diyagramında analiz, gereksinimlerin belirlenmesi, fizibilite, toplantılar, nesneye dayalı modelleme ve diyagramlar, bilet satış sisteminin gerçeklenmesi gibi iş paketleri vardı. Hoca süreç modelini sordu; grup çevik (agile) modeli seçtiğini söyledi. Hoca bunun zaman planında görünüp görünmediğini sordu ve Gantt diyagramını büyütmelerini istedi.

> **Hocanın vurgusu:** Çevik model seçince Scrum’daki sprintler gibi parça parça ilerleyen bölümleri belirlemeniz gerekir. Şelaledeki gibi “analiz bitti, tasarım başladı, tasarım bitti” diye sıralarsanız bu çevik model olmaz.

Grup, iş paketlerinin kısa kısa toplantılar ve kısa süreli işler şeklinde planlandığını, sürelere bakılınca bunun görüleceğini söyledi. Gantt diyagramı Excel’de çizilmişti.

> **Ek bilgi (derste anlatılmadı):** Scrum’da iş, genellikle 1-4 hafta süren sabit uzunlukta sprintlere bölünür. Her sprint planlama toplantısıyla başlar, sonunda çalışan bir ürün parçası gösterilir (sprint review) ve süreç gözden geçirilir (retrospective). Bu yüzden çevik bir projenin Gantt diyagramında analiz, tasarım, kodlama ve test her sprintin içinde tekrar eder; tek bir uzun analiz ve ardından tek bir uzun tasarım aşaması görünmez.

6.3 Risk Analizi

Hoca görev dağılımını geçmelerini, risk analizini anlatmalarını istedi. Üç risk anlatıldı:

- **Ödeme işlemlerinin güvenliği:** Acente nakit, kredi kartı ve çekle ödeme kabul ediyor; her birinin güvenlik zorlukları farklı. Olasılık düşük, etki yüksek. Önlem: özellikle kredi kartı ödemelerinde PCI DSS uyumlu biçimde işlem yapmak ve veriyi saklamak.
- **Veri güvenliği:** Müşteri bilgileri, bilet ve rezervasyon bilgileri gibi hassas veriler sistemde tutulur. Önlem: zararlı yazılımlara karşı koruyucu yazılımlar kullanmak ve düzenli veri yedeklemesi yapmak.

- **Yasal uygunluk:** Bilet satışı ve tatil rezervasyonu ticari işlemler olduğu için yasal düzenlemelere uyulmalıdır. Risk, acentenin bu düzenlemelere uymaması ya da yanlış uygulamasıyla ortaya çıkar.

6.4 Use Case, İzlenebilirlik ve Durum Diyagramı

Use case diyagramında yönetici, çalışan ve müşteri aktörleri ile yöneticinin özel işleri (puan oranını değiştirme gibi), sefer bileti satışı ve otel rezervasyonu use case'leri vardı. Ödeme use case'ine puan indirimi gibi özellikler bağlanmıştı. İzlenebilirlik tablosu modül bazında hazırlanmış ve her modülün hangi sınıfları içerdiğini gösteriyordu.

Durum diyagramı görevlinin ekranı üzerinden kurulmuştu:

```
[Sistem giriş ekranı] --kapat--> [Son]
[Sistem giriş ekranı] --giriş--> [Görevli ekranı]
[Görevli ekranı] --müşteri talebi--> [Talep kontrolü]
[Talep kontrolü] --uygun bilet/sefer yok--> [Görevli ekranı]
[Talep kontrolü] --bulundu--> [Şirkete bildirim]
[Şirkete bildirim] --uygun değil--> [Görevli ekranı]
[Şirkete bildirim] --uygun--> [Ödeme bekleme]
[Ödeme bekleme] --ödeme sorunu--> [Görevli ekranı]
[Ödeme bekleme] --ödeme tamam, şirket onayı--> [Bilet basıldı / satış yapıldı]
```

Etkinlik diyagramında görevli sisteme girer, müşteri acenteye gelip istediği hizmeti, tarihleri ve bilgileri verir. Müşterinin kayıtlı olup olmadığına bakılır, değilse kayıt yapılır. Görevli uygun tarih ve bölgedeki seçenekleri sunar, müşteri oteli seçer ve puanı olup olmadığı kontrol edilir.

6.5 Sıralama Diyagramları ve Tasarım Kalıpları

Sıralama diyagramları her kontrol (controller) sınıfı için ayrı çizilmişti; beş altı kadar kontrol sınıfı vardı. Örnekte müşteri kontrolcüsünden ödeme kontrolcüsüne geçiliyor, müşteri bilgileri servis üzerinden alınıyordu. Grup müşteri servisinin gerçeklemesinde (CustomerServiceImpl) tasarım kalıbı kullandıklarını söyledi.

Veritabanı SQLite ile kurulmuştu. Gerekçe, uygulamanın bütün bilgisayarlarda çalışabilmesiydi. ER diyagramını otomatik çıkarmak için aynı şema PostgreSQL'de de kurulmuştu. Tablolar şunlardı: ulaşım firması tipleri (otobüs, uçak), firmalar (ad, telefon), firmaların mevcut biletleri, müşteriler ve ödemeler.

Hoca tasarım kalıplarına hızlıca bakmak istedi. Grup şu kalıpları kullanmıştı:

- **Singleton:** Veritabanı bağlantısı (Connection) için.
- **DAO (Data Access Object):** Veritabanı ile program arasındaki erişim için. SQL işlemleri bir arayüzde (interface) tanımlanıyor, bir sınıf bu arayüzü gerçekleyip SQL'i yazıyordu.
- **Servis katmanı:** DAO ile arayüz kodları arasına bir servis katmanı konmuştu. Grup bunun gerekçesini testlerin daha yönetilebilir olması ve programın bakımının (maintenance) kolaylaşması diye açıkladı.
- **MVC (Model-View-Controller):** Arayüz için. Grup içinde Observer kalıbının da bulunduğunu söyledi.

Kod örneğinde otel kontrolcüsü önce bütün servis nesnelerini oluşturuyordu. Arayüzden seçilen bölgeye göre şehirler servis katmanından çekilip bir hash map'e dolduruluyor, sonra seçilen şehre göre otel servisinden sorgu yapılıyordu. Kod asistana önceden gösterildiği için hoca arayüz gösterimini atladı.

> **Ek bilgi (derste anlatılmadı):** Singleton, bir sınıftan uygulama boyunca tek bir nesne üretilmesini garanti eder; veritabanı bağlantısı gibi paylaşılan kaynaklar için sık kullanılır. DAO, veritabanı erişim kodunu iş mantığından ayıran bir arayüz katmanıdır. Servis katmanı DAO'yu bir arayüz üzerinden kullandığında, testlerde gerçek DAO yerine sahte bir gerçekleme verilebilir. Grubun "testler daha yönetilebilir olur" gerekçesi büyük olasılıkla buna dayanır.

Sunumdan sonra grup testlerini gösterme fırsatı olmadığını belirtti. Hoca gerek olmadığını söyledi.

7. Altıncı Sunum: İnsan Kaynakları Sistemi (include/extend ve Etkinlik Diyagramı Soruları)

7.1 Kabuller, Zaman Çizelgesi ve Süreç Modeli

Altıncı grup iki kişilik bir ekipti ve insan kaynakları vakasını sundu. Gereksinimler çalışan bilgilerini tutmak, çalışanları uygun projelere yerleştirmek, maaş ve tazminat işlemlerini yapmaktı. Kabullerde uygun proje varsa çalışan oraya atanıyor, proje bitiminde uygun proje bulunamazsa çıkarma işlemi yapılıyordu. Tazminat hesabı için sabit bir program, maaş için çalışanın iş yılına göre seçilen bir muhasebe programı kullanılıyordu. Hoca bunların vakadakilerden farklı, gruba özgü kabuller olup olmadığını sordu.

Zaman çizelgesinde sistemin değerlendirilmesi, iş tanımı, kapsamın belirlenmesi, diyagramlar, testler ve kodlama vardı. Hoca süreç modelini sordu; grup çevik (agile) bir yaklaşım düşündüklerini söyledi.

Risk tablosunda her aşama için olasılık belirlenmişti. Kullanım senaryosu örneği **yeni çalışan alımı** idi: İK çalışan bilgisini girer, uygun projeyi seçer, maaşın hangi programla hesaplanacağını seçer. Alternatif akışta uygun proje bulunamazsa işe alım süreci durdurulur.

7.2 include ve extend Oklarının Yönü

Use case diyagramında çalışan ve insan kaynakları aktörleri vardı; grup hem include hem extend kullanmıştı. Hoca diyagramda iki okun da aynı yönde çizildiğini gördü ve sordu:

Soru: Extend de kullanmışsınız, include da. İkisinde de okun yönü aynı mı olur, farklı mı? Okun yönü önemli mi? **Cevap (öğrenci):** Önemli. Extend ana use case'den başka bir use case'e doğru olur; include ise başka bir use case'in ana use case'e dahil edilmesidir.

Hoca "yani okun yönü nasıl olmalı, ikisi de aynı mı olur?" diye tekrar sorunca grup diyagramda bir hata olduğunu, okların ters çizildiğini kabul etti.

> **Ders sonrası not:** Öğrencinin cevabında yönler ters söylendi. UML'de include oku temel (ana) use case'den dahil edilen use case'e doğru çizilir: temel use case, dahil edilen davranışı her zaman kullanır. Extend oku ise genişleten (isteğe bağlı) use case'den temel use case'e doğru çizilir: genişleten davranış yalnızca belli bir koşulda temel use case'e eklenir. İki ok da kesikli çizgiyle ve `<<include>>` / `<<extend>>` etiketiyle gösterilir. Yani iki okun yönü birbirinin tersidir.

Hoca ardından use case belgesini bir kez daha görmek istedi ve grup gösterdi.

7.3 İzlenebilirlik, Sınıf ve Sıralama Diyagramları

İzlenebilirlik tablosunda her sınıf için hangi yapıların ve fonksiyonların hangi diğer sınıflarla ilişkili olduğu gösterilmişti.

Sınıf diyagramında bir ana insan kaynakları sınıfı vardı. Bu sınıf maaş (payroll) ve tazminat sınıflarıyla bileşim ve ilişki (association) bağlarına sahipti; maaş ve tazminat hesabında bu sınıflara erişiliyordu. Bunlar üzerinden birinci veya ikinci muhasebe programına ya da tazminat servisinin URL'sine ulaşılıyordu. İnsan kaynakları sınıfı proje sınıfıyla da bileşim ilişkisi içindeydi.

Hoca tasarımda kalıp olup olmadığını sordu. Grup, çalışan (Employee) sınıfını oluştururken **Factory** kalıbına benzer bir yapı kurmaya çalıştıklarını söyledi: soyut (abstract) bir Employee sınıfı doğrudan kullanılmıyor, ondan türeyen sınıflar onun işlevlerini kullanarak oluşturuluyordu.

Sıralama diyagramı yeni çalışan alındığındaki akışı gösteriyordu: proje kurucusu (constructor) çağrılır, kimlik gibi atamalar yapılır, bir takım atama (assign team) işlemiyle takım belirlenir, ardından Employee kurucusu çalışır ve çalışan ilgili takıma atanır.

7.4 Etkinlik Diyagramının Özelliği

Hoca tasarımda başka diyagram olup olmadığını sordu. Grup sıralama, etkinlik ve durum diyagramları olduğunu söyledi. Hoca etkinlik diyagramına geçmelerini istedi ve sordu:

Soru: Etkinlik diyagramının özelliği nedir? Bu diyagramda ne göstermiş oluyorsunuz? **Cevap (öğrenci):** Önce “her bir durum için kullanım durumları” dendi. Sonra başka bir sesle “aktivitelerde paralel akış için” diye eklendi.

> **Ders sonrası not:** “Her bir durum için kullanım durumları” tanımı yanlıştır. Bu tanım ne durum diyagramına ne use case’e karşılık gelir. Hocanın birinci sunumda verdiği tanım geçerlidir: etkinlik diyagramı bir sürecin akışını, sırasını ve koşullarını, özellikle de farklı ortamlarda yürüyen paralel akışların nerede ayrılıp nerede birleştiğini gösterir. Sonradan eklenen “paralel akış” cevabı doğru yöndedir.

Grubun etkinlik diyagramı iki senaryodaki kontrolleri gösteriyordu.

7.5 Durum Diyagramı ve Testler

Durum diyagramında iki döngü vardı. Birincisinde personel alındığında uygun projeye atanıyor, proje açılıyor ve proje kapasitesi sürekli kontrol ediliyordu; kapasite dolana kadar işlem devam ediyor, uygun proje kalmayınca bu durumdan çıkılıyordu. İkincisinde biten proje aranıyordu: proje görüntüleniyor, bitiş süresi kontrol ediliyor, proje devam ediyorsa döngü sürüyordu. Proje bittiyse çalışanlar projeden çıkarılıyor, bilgileri alınıyor, uygun proje varsa yeni projeye atanıyor, yoksa tazminat hesaplanıp işten çıkarılıyordu.

Hoca test sorduğunda grup birim testlerini yetiştiremediğini söyledi.

8. Yedinci Sunum: Medya Takip Sistemi (Abonelik ve Onay Akışı)

8.1 Kabuller ve Kısıtlar

Yedinci grup, bazı üyeleri projeye katılmadığı için üç kişi tamamladıkları medya takip sistemini sundu. Grup sistemin özelliklerini saymaya başlayınca hoca alan tanımını bildiklerini, yalnızca gruba özgü kısıtları anlatmalarını istedi. Grubun söyledikleri şunlardı:

- Sisteme müşteri ve sistem yöneticisi olmak üzere iki farklı rolle giriş yapılır.
- Müşteriler kişisel ya da şirket profiliyle üye olabilir.
- Müşterinin hizmetten yararlanması için abonelik tipini seçip ücreti ödemesi gerekir.
- Sistem yöneticisi kaydı onayladıktan sonra müşteri kullanıcı adı ve parolasıyla sisteme bağlanabilir.

8.2 Zaman Çizelgesi ve Risk

Proje araştırmayla başladı, ardından finansal ve teknik değerlendirme yapıldı, sonra kodlama ve test geldi. Hoca süreç modelini sordu; grup şelale modelini kullandığını söyledi.

Risk tablosu kimliklere göre düzenlenmiş ve riskler “teknik” ve “proje” diye gruplanmıştı. Hoca önlemleri düşünüp düşünmediklerini sordu. Grup kendi yaşadığı bir riski örnek verdi: proje beş kişiyle başlamıştı, iki kişinin katılmaması geç teslim riskini doğurdu. Önlem olarak projeyi diğer derslerinin ve projelerinin önüne alıp öncelik verdiler. Bu, hocanın daha sonra dokuzuncu sunumda istediği türden projeye özgü bir risktir.

8.3 Abonelik Senaryosu ve Diyagramı

Beş kullanım senaryosu vardı. Hoca bir tanesini seçip anlatmalarını, sonra onun diyagramını göstermelerini istedi. Grup **müşteri aboneliği** senaryosunu anlattı:

- Müşteri kayıt sırasında abonelik tipini ve ilgilendiği medya tiplerini seçer.
- Günlük, haftalık veya aylık rapor seçeneklerinden birini belirler.
- Sistem onayından sonra ödemesini yapar ve hizmet almaya başlar.

Diyagramda kullanıcı kayıt ekranına bilgilerini girer, üyelik tipini (şahıs veya şirket) seçer, rapor periyodunu seçer. İsterse standart aralıklar yerine kendine özel bir rapor aralığı belirleyebilir. Ödemeden sonra sistem yöneticisi kaydı onaylar ya da reddeder. Hoca senaryoda include’ların olduğunu gördü ve extend olup olmadığını sordu; grup extend olmadığını söyledi. Hoca bir senaryoyu görmenin yeterli olduğunu belirtti.

8.4 İzlenebilirlik ve Sınıf Diyagramlarının Aşaması

İzlenebilirlik tablosunda veritabanı servisi bütün modüller için, arayüz tasarımı giriş, anket, haber ve kullanıcı modülleri için sağlanıyordu. KVKK izinlerinin alınması giriş modülünde tamamlanıyordu.

Grup ardından iki diyagram gösterdi: nesneye dayalı modelleme şeması ve altında metotların gösterildiği ikinci bir diyagram. Hoca sordu:

Soru: Bu iki diyagramdan biri hangi aşamada, diğeri hangi aşamada kullanılıyor? **Cevap (öğrenci):** İlk gösterilen kavramsal sınıf diyagramı, tasarım aşamasında kullanılır. İkincisi analiz aşamasında kullanılır; o da tasarım sınıf diyagramıdır.

Hoca "bana bu cevap yeterli" deyip geçti.

> **Ders sonrası not:** Öğrenci aşamaları ters söyledi. Kavramsal sınıf diyagramı analiz aşamasında çizilir; alan kavramlarını ve aralarındaki ilişkileri gösterir, metotlar genellikle yoktur. Tasarım sınıf diyagramı tasarım aşamasında çizilir; metotlar, görünürlük, veri tipleri ve kullanılan tasarım kalıplarıyla tamamlanmış hâlidir. Grubun gösterdiği sıra (önce metotsuz model, sonra metotlu model) bu doğru sıralamaya uyuyordu; yalnızca aşama adları ters söylendi.

8.5 Sıralama, Etkinlik ve Durum Diyagramları

Sıralama diyagramı kodun akışını anlatmak için çizilmişti. Etkinlik ve durum diyagramları beş senaryonun her biri için ayrı ayrı hazırlanmıştı.

Müşteri aboneliği etkinlik diyagramı: müşteri kayıt ekranına gider, bireysel ya da şirket kullanıcı tipini seçer, standart ya da özel periyotlu rapor tipini seçer, ödeme yapar. Sistem tarafında oturum açmış bir yöneticinin yeni kaydı onaylaması ya da reddetmesi gerekir; bununla etkinlik tamamlanır.

Durum diyagramı müşteri kaydının durumlarını gösteriyordu:

```
[Kayıt ekranında] --form gönderildi--> [Onay bekliyor]
[Onay bekliyor] --yönetici onayladı--> [Onaylandı] (sistemi kullanabilir)
[Onay bekliyor] --yönetici reddetti--> [Reddedildi]
```

8.6 Birim Testleri

Üç kişi kişi başına iki test yazarak altı birim testi hazırlamıştı; hepsi sorunsuz çalışıyordu. Anlatılan örnekte bir kişi adı, kişilik tipi (kişi veya şirket) ve üyelik tipi (günlük, haftalık, aylık veya özel) veriliyordu. Test, girilen kullanıcının üyelik tipinin beklenen tiple (örnekte aylık) aynı olup olmadığını kontrol ediyor; aynıysa test geçiyor, değilse `false` dönüyordu.

9. Sekizinci Sunum: İnsan Kaynakları Sistemi (Başvuru Odaklı Tasarım)

9.1 Kabuller, Plan ve Risk

Sekizinci grup insan kaynakları vakasını dört kişi olarak sundu. Vakadaki kabulleri tekrarlamadılar, şunları eklediler:

- Dört rol vardır: yönetici, analist, programcı ve tasarımcı.
- **Kabul:** Her projede bir tane yönetici bulunur.
- **Kısıt:** İşten çıkarmalar yalnızca proje bitiminde gerçekleşir; projede çalışan biri işten çıkarılmaz.

İş zaman çizelgesi dört kişilik ekip için iki aylık bir süreyi kapsıyordu. Gantt diyagramında analiz ve tasarım, programlama, test, uygulama ve entegrasyon, test ve kabul, dokümantasyon aşamaları vardı. Grup şelale modelini kullandı.

Risk tablosunda beş risk; öncelik, olasılık ve etkiye göre sınıflandırılmış, her biri için önlem ve sorumlu kişi yazılmıştı. Örnek: **yazılım hatası** yüksek öncelikli bir risk; olasılığı orta, etkisi yüksek. Önlem olarak kod incelemesi ve test planı hazırlanmıştı.

9.2 Use Case ve Sınıf Diyagramı

Örnek kullanım senaryosu İK'nın yeni bir çalışan almaya karar vermesiydi ve adımlar yazılmıştı. Use case diyagramında iki aktör vardı:

- **Çalışan** (analist, tasarımcı, programcı ya da yönetici olabilir): proje bilgilerini ve çalışan bilgilerini görüntüler.
- **Yönetici:** çalışan tazminat bilgisini, proje bilgilerini ve çalışan bilgilerini düzenler.

Görevlerle ilgili bir izlenebilirlik tablosu da hazırlanmıştı. Nesneye dayalı modelde Projeler, şirketin İK sistemi sınıfı, Çalışan ve Tazminat sınıfları vardı; aralarında kalıtım yoktu. Tasarımda bu temel sınıflara metotlar eklenmişti.

9.3 Durum, Sıralama ve Etkinlik Diyagramları

Durum diyagramı iş başvurusuyla başlıyordu:

```
[Başvuru yapıldı] --> [Reddedildi]
[Başvuru yapıldı] --> [Kabul edildi]
[Kabul edildi] --proje değerlendirmesi--> [Projeye atandı]
[Projeye atandı] --> [Projede çalışıyor]
[Projede çalışıyor] --proje bitti--> [Ödeme] veya [Yeni proje başvurusu]
```

Sıralama diyagramında kullanıcı, insan kaynakları, proje ve hesap nesneleri vardı:

```
Kullanıcı -> İK      : iş başvurusu
İK -> Proje          : çalışan eksiği var mı?
Proje -> Hesap       : bütçe sorgusu
Hesap -> Proje       : bütçe bilgisi
Proje -> İK          : (uygunsa) çalışan açığı bilgisi
İK -> Kullanıcı      : başvuru kabul / red
Kullanıcı -> Proje   : projede çalışma
Proje -> İK          : çalışma sonucu bilgisi
Hesap -> Kullanıcı   : ödeme
```

Etkinlik diyagramında başvuru alınır, çalışan eksiği olup olmadığı sorgulanır. Eksik varsa uygun proje atanır, yoksa başvuru reddedilir. Uygun proje yoksa çalışan bekletilir, varsa proje tamamlanır ve ödeme yapılır; sonra yine çalışan eksiğine göre akış sürer.

Grup birim testlerini yetiştiremediğini, kodu ise asistana daha önce gösterdiğini söyledi. Hoca grubun use case diyagramı, belgesi ve tasarım diyagramlarının bulunduğunu not etti.

10. Dokuzuncu Sunum: İnsan Kaynakları Sistemi (Milestone ve Projeye Özgü Risk)

10.1 Gruba Özgü Kabuller ve Kısıtlar

Dokuzuncu grup da insan kaynakları sistemini sundu. Hoca yine yalnız gruba özgü kabul ve kısıtları istedi. Grubun kabulleri şunlardı:

- Yönetici yeni bir proje oluşturduğunda proje yöneticisini kendisi seçer.
- Yeni proje oluşturulduğunda projenin gereksinimleri, örneğin en az çalışan sayısı, belirlenir.
- İşe alımda çalışan bilgileri yüz yüze paylaşılır.
- İşten çıkarmada ödenecek tazminat için bir web servis programı kullanılır. (Hoca bunun vakada zaten bulunduğunu belirtti.)

Kısıtları ise doğrudan veri doğrulama kurallarıydı:

- Çalışan eklenirken geçersiz bir e-posta adresi girilemez.
- Ad, soyad, e-posta gibi önemli alanlar boş bırakılamaz.
- İş tanımındaki en düşük maaştan daha az maaş girilemez.

10.2 Gantt Diyagramı ve Milestone

İş zaman çizelgesinde ön inceleme toplantısı, gereksinimlerin belirlenmesi, kullanım senaryoları, use case diyagramları ve sistem ile veritabanının hazırlanması vardı. Diyagram Excel'de çizilmişti. Hoca sordu:

Soru: Milestone'unuz var mı? Göremiyorum. **Cevap:** Grup milestone olmadığını söyledi ve terimin ne demek olduğunu sordu.

Hoca milestone'u şöyle tanımladı: işin (özelliğin) belli bir yere gelmesi ve o noktada toplantıların yapılması.

> **Hocanın vurgusu:** Gantt diyagramında milestone'lar gösterilmelidir. Milestone, işin belirli bir noktaya ulaştığı ve bir toplantı ya da değerlendirmenin yapıldığı yerdir.

Ekip organizasyonu proje yöneticisi, iş analistleri ve yazılımcılardan oluşuyordu.

10.3 Risklerin Projeye Mal Edilmesi

Risk tablosunda örnek olarak iki risk anlatıldı. Birincisi proje takviminin gecikmesiydi; önlem olarak Gantt diyagramında çok sıkı zaman aralıkları belirlenmemiş, pay bırakılmıştı. İkincisi bir çalışanın işten ayrılmasıydı; önlem olarak hızlıca yeni birinin gruba alınması öngörülmüştü. Tabloda ayrıca elektrik kesintisi ve veri güvenliği gibi riskler de vardı.

Hoca tabloda etki, olasılık ve risk türünün belirlenmiş olmasını olumlu buldu, ama risklerin seçimini eleştirdi:

> **Hocanın vurgusu:** Bu bir dönem projesi. Elektrik kesintisi kısa süreli bir şeydir; veri güvenliği de sizi çok ilgilendiren bir risk değil. "Çalışanın işten ayrılması" riskini kendinize göre güncelleyebilirdiniz: "ekipten bir kişinin dersi bırakması" ya da "ekipten birinin bilgi eksikliği" diyebilirdiniz. Riskleri biraz kendinize mal etmelisiniz.

Öğrenci, tabloyu daha "profesyonel" bir proje gibi mi yoksa bir ders projesi gibi mi yazmaları gerektiğini düşündüğünü söyledi. Hocanın cevabından, risklerin ekibin gerçekten yürüttüğü projeye göre yazılması gerektiği anlaşılır.

10.4 Kullanım Senaryosu ve Sınıf Diyagramı

Grup **işe alım** senaryosunu anlattı. Yönetici işe alım yapar, çalışanın bilgilerini tek tek sisteme girer ve çalışan uygun bir projeye atanır. Atanabileceği uygun proje yoksa çalışan işe alınamaz. Hoca diğer senaryolara geçmek yerine bu senaryonun diyagramını görmek istedi. Diyagramda yönetici çalışan bilgisini girerken maaş fonksiyonunu da belirliyor, çalışan uygun projeye atanıyor ve yönetici çalışan bilgilerini görüntüleyebiliyordu.

Hoca ardından veri modelini, yani sınıf diyagramını istedi ve sınıf diyagramının ne zaman çizildiğini sordu. Grup "analiz aşamasında" diye cevap verdi. Hoca bir tasarım kalıbı olup olmadığını ya da yalnız var olan sınıflarla mı çizdiklerini sordu ve tasarım diyagramını büyüterek göstermelerini istedi. Grup tasarım diyagramına metotları eklediklerini gösterdi.

10.5 Sıralama, Durum ve Etkinlik Diyagramları

Sıralama diyagramında iki aktör vardı: direktör ve proje yöneticisi. İkisi de arayüz üzerinden sistemle iletişim kuruyordu. Direktör proje oluşturabiliyor, işe alım yapabiliyor ve projeyi sonlandırabiliyordu. Proje yöneticisi kendi projesine çalışan ekleyip çıkarabiliyordu. Arayüz nesnesi istekleri sunucuya iletiyor, ilgili değişiklikler projede yapılıyordu.

Durum ve etkinlik diyagramları projeler için görevlendirmenin nasıl yapıldığını anlatıyordu. Durum diyagramında yazılım evinin elindeki projeler listeleniyor, her proje için bir proje yöneticisi görevlendiriliyordu. Proje yöneticisi kapsamına göre gereken çalışan sayısını belirliyor, çalışanları listeleyip uygun bulduklarını projeye atıyordu. Çalışan sayısı yetmezse yeni çalışan alınıyor ve maaşı belirleniyordu. Hiçbir projeye uygun bulunamayan personelin tazminatı hesaplanıp işine son veriliyordu. Etkinlik diyagramı şirket yeni bir proje aldığında izlenen akışı

gösteriyordu: proje oluşturulur, yönetici atanır, gereken çalışan sayısı belirlenir, çalışan bilgileri listelenir.

10.6 Birim Testleri

Grup on test metodu yazmıştı:

- İki maaş hesaplama metodunun doğru çalışıp çalışmadığı.
- Çalışan, proje, çalışanların projeleri ve işler (job) listelerinin doğru gelip gelmediği.
- Çalışan silindiğinde doğru silinip silinmediği.
- Çalışanın işi değiştiğinde ve projenin yöneticisi değiştiğinde değişikliğin doğru yapılıp yapılmadığı.
- Modelin doğru çalışıp çalışmadığı.

Testler çalıştırıldı ve hepsinin başarıyla geçtiği gösterildi. Grup izlenebilirlik tablosunu gösteremediğini söyledi; hoca raporda bakacağını belirtti.

11. Onuncu Sunum: Seyahat Acentesi (PostgreSQL ve PyTest)

11.1 Kabuller, Plan ve Ekip

Son grup seyahat acentesi vakasını sundu. Alan tanımını geçip gruba özgü kabullere geldiler:

- Otel ve ulaşım fiyatlarının süreç boyunca sabit kaldığı varsayıldı.
- Sistemi bir sistem operatörü kullanır; uygulama bir masaüstü uygulamasıdır.
- Veritabanı olarak SQL tabanlı PostgreSQL seçildi; gerekçe ekibin tecrübesiydi.
- Kullanıcıların KVKK kapsamında kişisel bilgilerinin veritabanına kaydedilmesine izin verdiği varsayıldı.

Gantt diyagramı çevrim içi bir Gantt aracıyla çizilmişti. Ön incelemede herkes görev aldı. Ardından sistem analizi ve veritabanı tasarımı, sonra backend, sonra frontend geldi; en sonda testler yazılıp kodlar birleştirildi.

11.2 Risk Tablosu ve Risk Matrisi

Grup riskleri olasılık ve etkiye göre değerlendirdi:

Risk	Olasılık	Etki
Yazılımın hiç tamamlanmaması veya teslim edilmemesi	Düşük	Çok yüksek
Projenin tasarım aşamasından sonra iptal edilmesi	Düşük	Çok yüksek
Yazılımın geç teslim edilmesi	Görece daha olası	Orta
Geliştirme bütçesinin %20'ye kadar aşılması	Oldukça olası	Düşük
Geliştirme bütçesinin %20'den fazla aşılması	Düşük	Yüksek
Bakım maliyetlerinin uzun vadede beklenenden yüksek olması	Düşük	Düşük
Zaman hedeflerinin karşılanamaması	Orta (ekibin tecrübesizliği nedeniyle)	Düşük

Bütçe riski iki kademede ele alınmıştı: küçük bir aşımın olasılığı yüksek ama etkisi düşük; aşım %20'yi geçince olasılık düşer ama etki artar. Grup, bir ders projesinde gerçek bir bütçe olmasa da bir projede geliştirme bütçesinin bulunacağını belirtti.

Grup riskleri bir olasılık-etki matrisine yerleştirdi. 1 ve 2 numaralı risklerin etkisi çok yüksek ama olasılığı çok düşüktü; 4 ve 7 numaralıların olasılığı görece yüksek ama etkisi çok düşüktü. Bu yüzden projenin kabul edilebilir bir risk aralığında olduğu sonucuna vardılar.

> **Ek bilgi (derste anlatılmadı):** Olasılık-etki matrisi (risk matrisi), her riski olasılık ve etki eksenlerinde bir hücreye yerleştirir. İkisi de yüksek olan riskler öncelikli olarak ele alınır. Etkisi yüksek ama olasılığı düşük riskler için genellikle bir acil durum planı hazırlanır. Olasılığı yüksek ama etkisi düşük riskler ise çoğu zaman izlenir ya da kabul edilir. Birinci sunumda hocanın söylediği "olasılık çarpı etki" değeri bu matristeki konumun sayısal karşılığıdır.

11.3 Senaryo ve Use Case Diyagramı

Senaryoda iki taraf vardı: yolcu ve acente görevlisi.

- **Ön koşul:** Yolcu seyahat tarihlerini ve yolculuk planını belirlemiştir.
- **Son koşul:** Yolcunun seyahat planı veritabanına kaydedilmiş ve rezervasyon tamamlanmıştır.
- **Ana akış:** Yolcu acente görevlisine gelir, tarihi ve istediği rezervasyonu belirtir. Görevli ilgili ulaşım ve konaklama bilgilerine göre müsaitliği kontrol eder; rezervasyon ve ödeme tamamlanır.
- **Alternatif akış:** Yolcunun birikmiş bonus puanlarıyla ödeme yapmak istemesi ya da görevlinin yolcuya başka seçenekler sunması.

Use case diyagramında müşteri istediği tarih ve özelliklere göre bilet arar; acente görevlisi ulaşım ve otel bilgilerini bu tarihlere göre sunar. Ödeme yöntemi alternatif olarak bonus puanlarla yapılabilir; bilet alımlarında da bonus puan birikir.

Gereksinim ve test senaryoları tablosunda örnek olarak müşterinin bir otelden konaklama seçmesi vardı: oteller listelenir, seçilen tarihteki müsaitliğe göre seçilen otel müşterinin adına veritabanına kaydedilir. Ulaşım seçenekleri için de aynı yapı vardı.

11.4 Sınıf Diyagramı

UML sınıf diyagramında oteller ve ulaşım firmaları (örneğin demiryolu işletmesi) vardı. Bunların birçok otel rezervasyonu ya da ulaşım seçeneği (transport option) bulunuyordu. Müşteri bir rezervasyon (booking) yaptığında Booking nesnesi şu bilgileri tutuyordu:

- Müşteri kimliği (customer ID).
- Rezervasyonun otel mi ulaşım mı olduğu ve ilgili nesne.
- Başlangıç ve bitiş tarihleri, kişi sayısı.
- Toplam fiyat (total price) ve otele göre hesaplanan toplam bonus (total bonus).

Bu bilgiler veritabanında tutuluyor ve rezervasyon sistemi Booking üzerinden işliyordu.

11.5 Etkinlik Diyagramı ve Ortam Sorusu

Etkinlik diyagramında müşterinin iki seçeneği vardı: ulaşım ve konaklama. Ulaşım seçilirse seçenekler bulunur, bir seçenek seçilip seçilmediği kontrol edilir, seçildiyse ulaşım rezervasyonu yapılır. Konaklama için aynı adımlar vardır. Ödemeye gelindiğinde müşterinin bonus puanı olup olmadığı kontrol edilir; varsa puan uygulanır, yoksa doğrudan ödemeye geçilir. Sonunda müşteriye onay mesajı gönderilir.

Soru: Ulaşım ve konaklama birimi ayrı ayrı mı? Yani ayrı alanlarda gerçekleşen işlemler değil mi? **Cevap:** Evet, ulaşım birimi ve konaklama birimi ayrıdır.

Hocanın sorusu, etkinlik diyagramında farklı birimlerin farklı ortamlar (kulvarlar) olarak gösterilmesi gerektiğine yönelikti.

11.6 PyTest ile Birim Testleri

Birim testleri Python'da PyTest ile yazılmıştı. Grup, PyTest'te Java'daki Mockito gibi bir yapı bulunmadığını, bu yüzden testlerin gerçek ana veritabanıyla iletişim kurduğunu ve gerçek veri

eklediğini söyledi. Veritabanını özgün hâlinde tutmak için test sırasında oluşturulan kayıtlar saklanıyor ve test sonunda siliniyordu.

Müşteri, ulaşım firması, otel ve rezervasyon için CRUD işlemleri (create, update, get, delete) test edilmişti. Örneğin "bir ulaşım firmasına ait bütün otelleri/ulaşımları getir" fonksiyonu, firma kimliğine göre bir liste döndürüyordu. Test, repository'deki fonksiyonun aynı sorguyla aynı sonucu verip vermediğini kontrol ediyordu. Silme işlemleri de aynı biçimde test edilmişti.

> **Ders sonrası not:** Grup "PyTest'te Mockito gibi bir yapı yok" dedi. Python'un standart kütüphanesindeki `unittest.mock` modülü ve PyTest için `pytest-mock` eklentisi aynı işi görür. Veritabanı erişimi mock'lanarak testler gerçek veriden bağımsız yapılabilir; böylece test sonunda kayıt silme gerekmez. İkinci sunumdaki `verify(..., times(1))` kontrolünün Python karşılığı `assert_called_once()` metodudur.

Sunumun sonunda bütün grupların anlattığı kontrol edildi.

12. Sunumlardan Çıkan Ortak Dersler

On sunum boyunca hocanın soruları birkaç temel noktada toplandı. Bunlar dönem projesinin hangi belgelerinin nasıl hazırlanması gerektiğini özetler:

- **Kabuller ve kısıtlar:** Vakada verilenler tekrar edilmez, gruba özgü olanlar yazılır. Kabuller ve kısıtlar ayrı başlıklarda tutulur.
- **Süreç modeli:** Her grup hangi modeli seçtiğini bilmeli. Sorulan gruplardan çoğu şelale, ikisi çevik modeli seçti. Çevik model seçildiyse Gantt diyagramında sprintler görünmelidir.
- **Gantt diyagramı:** Milestone'lar, yani işin belli bir noktaya geldiği ve toplantı yapılan yerler gösterilmelidir.
- **Risk tablosu:** Olasılık, etki ve bunlardan çıkan önceliğin yanında önlem de yazılır. Riskler projeye özgü olmalıdır.
- **Use case:** Diyagramın yanında açıklamalı use case belgesi bulunmalıdır. include ve extend oklarının yönleri terstir.
- **Sınıf diyagramları:** Kavramsal sınıf diyagramı analizde, tasarım sınıf diyagramı tasarımda kullanılır.
- **Etkinlik diyagramı:** Akışı, koşulları, paralel kolları ve farklı ortamları (kulvarları) gösterir.
- **Tasarım kalıpları:** Gruplarda Singleton, DAO ve servis katmanı, MVC ile Observer, Builder, Factory benzeri yapı ve controller-service-repository katmanları kullanıldı; bazı gruplar kalıp kullanmadı.
- **Birim testleri:** Mockito (Java), GoogleTest (C++) ve PyTest (Python) kullanıldı. İyi bir birim testi dış bağımlılıkları mock'layarak yalnızca test edilen metodu sınar.

Örnek Sorular

Bu derste ekranda soru gösterilmedi. Aşağıdaki sorular hocanın sunumlar sırasında gruplara sözlü olarak sorduğu teknik sorulardır.

Soru 1 (derste sözlü, 00:14:11): Bu gösterdiğiniz tasarım sınıf diyagramı mı, kavramsal sınıf diyagramı mı?

> **Çözüm:** Diyagramda yalnızca sınıflar, alt sınıflar ve öznitelikler vardı, metot yoktu. Grup diyagramın kavramsal olduğunu ve başlığı yanlış yazdığını kabul etti. Metotların eklendiği hâli tasarım sınıf diyagramıdır.

Soru 2 (derste sözlü, 00:17:43): Etkinlik diyagramı özellikle neyi ortaya koymak için çizilir?

> **Çözüm:** Etkinliğin akışını, sırasını ve koşullarını gösterir. Hocanın eklediği gibi, özellikle farklı ortamlarda (kişi, birim) yürüyen paralel akışların nerede ayrılıp nerede birleştiğini göstermek için kullanılır.

Soru 3 (derste sözlü, 00:57:52): Model olarak hangi süreç modelini seçtiniz?

Çözüm: Grup şelale modelini seçtiğini söyledi. Hocanın bu soruyu her grupta sormasının nedeni, Gantt diyagramının seçilen modele uygun olması gerektiğidir (bkz. Soru 7).

Soru 4 (derste sözlü, 01:03:29): Use case belgeniz var mı?

Çözüm: Use case belgesi, diyagramdaki her kullanım senaryosunun açıklamalı metin tanımıdır: birinci aktör, ilgililer ve beklentileri, ön koşul, son koşul, ana akış ve alternatif akışlar. Yalnızca diyagram göstermek yetmez.

Soru 5 (derste sözlü, 01:08:44): Bu tasarımda kullandığınız bir tasarım kalıbı var mı?

Çözüm: Bu grup kalıp kullanmamıştı. Diğer gruplarda verilen cevaplar: veritabanı bağlantısı için Singleton, veri erişimi için DAO ve servis katmanı, arayüz için MVC ve Observer, model nesneleri için Builder, çalışan alt sınıfları için Factory benzeri bir yapı.

Soru 6 (derste sözlü, 01:22:10): Etkinlik diyagramında bu işlerin ortamlarını belirtmemişsiniz; hangisi hangi ortamda oluyor?

Çözüm: Grup ortam ayrımı yapmamıştı. Etkinlik diyagramında her işin hangi ortamda (aktör ya da birim) yapıldığı kulvarlarla (swimlane) gösterilmelidir.

Soru 7 (derste sözlü, 01:30:16): Süreç modeliniz çevik ise bu zaman planınızda belli mi?

Çözüm: Çevik model seçilince Gantt diyagramında Scrum'daki sprintler gibi parça parça ilerleyen bölümler görünmelidir. Analiz bitti, tasarım başladı şeklinde ardışık aşamalar şelale modelinin görünümüdür.

Soru 8 (derste sözlü, 01:47:59): Extend ve include kullanmışsınız. İkisinde okun yönü aynı mı, farklı mı? Okun yönü önemli mi?

Çözüm: Okun yönü önemlidir ve iki ilişkide terstir. include oku temel use case'den dahil edilen use case'e, extend oku genişleten use case'den temel use case'e doğru çizilir. Gruptaki öğrenci yönleri ters söyledi; grup diyagramdaki okların hatalı olduğunu kabul etti (bkz. 7.2'deki ders sonrası not).

Soru 9 (derste sözlü, 01:53:07): Etkinlik diyagramının özelliği nedir?

Çözüm: Öğrencinin ilk cevabı ("her bir durum için kullanım durumları") yanlıştı; sonra "paralel akış için" diye eklendi. Doğru cevap Soru 2'deki tanımdır.

Soru 10 (derste sözlü, 02:01:58): Risklerinizin önlemlerini düşündünüz mü?

Çözüm: Grup kendi yaşadığı riski örnek verdi: iki üyenin katılmaması geç teslim riski doğurdu; önlem olarak projeye diğer derslerden önce öncelik verildi. Risk tablosunda her riskin önlemi de yazılmalıdır.

Soru 11 (derste sözlü, 02:04:31): Senaryonuzda include'lar var; extend var mı?

Çözüm: Grubun diyagramında extend yoktu. Extend, temel senaryoya yalnızca belli bir koşulda eklenen isteğe bağlı davranışlar için kullanılır.

Soru 12 (derste sözlü, 02:05:23): Gösterdiğiniz iki sınıf diyagramından biri hangi aşamada, diğeri hangi aşamada kullanılır?

Çözüm: Kavramsal sınıf diyagramı analiz aşamasında, tasarım sınıf diyagramı tasarım aşamasında kullanılır. Öğrenci aşamaları ters söyledi (bkz. 8.4'teki ders sonrası not).

Soru 13 (derste sözlü, 02:17:53): Gantt diyagramınızda milestone var mı?

Çözüm: Grubun diyagramında yoktu. Hocanın tanımıyla milestone, işin belli bir noktaya geldiği ve toplantıların yapıldığı yerdir; Gantt diyagramında gösterilmelidir.

Soru 14 (derste sözlü, 02:21:50): Sınıf diyagramı ne zaman çizilir?

Çözüm: Grup "analiz aşamasında" dedi. Bu cevap kavramsal sınıf diyagramı için doğrudur; metotlarla tamamlanan tasarım sınıf diyagramı tasarım aşamasında çizilir.

Soru 15 (derste sözlü, 02:49:48): Ulaşım ve konaklama birimi ayrı ayrı mı, yani ayrı alanlarda gerçekleşen işlemler değil mi?

> **Çözüm:** Grup birimlerin ayrı olduğunu doğruladı. Ayrı birimlerde yürüyen işler etkinlik diyagramında ayrı ortamlar (kulvarlar) olarak gösterilmelidir.

Dersin Toparlaması

Bu ders, dönem boyunca öğrenilen yazılım mühendisliği belgelerinin bir proje üzerinde nasıl bir araya geldiğini gösterdi. Gruplar aynı üç vakadan yola çıksa da kabul ve kısıtlarını, risklerini ve tasarım kararlarını farklı biçimlerde kurdu. Hocanın soruları bu belgelerin her birinin neden var olduğunu yokladı. Süreç modeli Gantt diyagramına yansımalı, Gantt diyagramında milestone'lar bulunmalı, risk tablosu projeye özgü olmalı ve önlem içermeli, use case diyagramı belgeyle desteklenmeli, include ve extend okları doğru yönde çizilmeli, kavramsal ve tasarım sınıf diyagramları doğru aşamaya yerleştirilmeli, etkinlik diyagramı paralel akışları ve ortamları göstermelidir.

Tasarım ve test tarafında katmanlı mimari (controller, service, repository), DAO ve servis katmanı, Singleton, Builder ve MVC gibi kalıpların gerçek projelerde nasıl kullanıldığı görüldü. Birim testlerinde ise dış bağımlılıkları mock'layarak yalnızca test edilen metodu sınamanın, gerçek veritabanına yazıp sonra temizlemeye göre daha sağlam bir yol olduğu ortaya çıktı. Ders, bütün grupların sunumu tamamlamasıyla ve sınavlar için başarı dileğiyle kapandı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.